راهنمای جامع و معرفی‌نامه رسمی OSS Architect Scout

موتور هوشمند ارزیابی، رتبه‌بندی و همرنگ‌سازی مخازن متن‌باز در معماری نرم‌افزار

نسخه: 1.0.0 | تست‌ها: ۳۰۶ پاس‌شده (۱۰۰٪) | معماری: Layered Architecture | مجوز: MIT

۱. فلسفه وجودی و مأموریت اصلی

در دوران مدرن توسعه نرم‌افزار و به ویژه در جریان **وایب‌کدینگ (Vibe-Coding)**، بزرگ‌ترین هدررفت زمان مهندسان، **«اختراع دوباره چرخ»** است. توسعه‌دهندگان ساعت‌ها وقت صرف پیاده‌سازی اجزایی می‌کنند که پیش‌تر توسط مجرب‌ترین مهندسان دنیا در پلتفرم گیت‌هاب پیاده‌سازی، تست و تثبیت شده است. از سویی دیگر، کپی کردن کورکورانه کدهای گیت‌هاب به درون پروژه، منشأ اصلی ورود باگ‌های امنیتی، تداخلات معماری و فجایع حقوقی نقض لایسنس است.

سیستم **OSS Architect Scout** در نقش یک **دستیار سیستمیک و معمار ارشد متن‌باز** عمل می‌کند تا این فرآیند را از ابتدا تا انتها مکانیزه کند:

- گیت‌هاب را به صورت زنده و با استفاده از API رسمی برای نیازمندی‌های پروژه کاوش می‌کند.
- کاندیداها را از یک قیف ۳ مرحله‌ای غربالگری پرسرعت عبور می‌دهد تا زمان پاسخ‌دهی به زیر ۱۰ ثانیه برسد.
- با فیلتر وتوی مطلق، پروژه‌های دارای آسیب‌پذیری‌های بحرانی یا لایسنس‌های ویروسی (مانند AGPL در پروژه‌های تجاری) را فوراً حذف می‌کند.
- پروژه‌های بازمانده را بر اساس یک ماتریس ۵ فاکتوری متوازن نمره‌دهی می‌کند.
- کدهای منتخب را به صورت کپسول‌های ماژولار و همراه با DTOها و آداپتورهای اختصاصی استانداردسازی و بسته‌بندی می‌نماید.

۲. راهنمای فراخوانی و استفاده

الف) فراخوانی محاوره‌ای در چت (Antigravity / Cursor / Claude Code)

تنها کافی است نیاز فنی خود را در پنجره چت مطرح نمایید:

```
@oss-architect-scout را ارزیابی و پیشنهاد کن FastAPI در JWT برای پروژه‌ام یک ماژول احراز هویت با
```

ب) اجرای خط فرمان (CLI)

```
# ارزیابی مخازن برای احراز هویت
python -m src.ui.cli_controller --query "jwt authentication fastapi" --path "."

# جستجو با امکان استفاده از کدهای کپی‌لفت (پروژه‌های غیرتجاری)
python -m src.ui.cli_controller --query "vector search engine" --allow-copyleft

# مشاهده تاریخچه ارزیابی‌های پیشین در دیتابیس
python -m src.ui.cli_controller --mode history
```

ج) استفاده مستقیم در کدهای پایتون (Programmatic API)

```
from src.core.container import Container

container = Container()
orchestrator = container.create_orchestrator()

result = orchestrator.scout(query="task queue redis", project_path=".")
print(result.report)
```

۳. جریان کارکرد قیف ۳ مرحله‌ای (3-Stage Pruning Funnel)

جهت بهینه‌سازی حداکثری سرعت و مصرف توکن‌ها، غربالگری به شیوه آبشاری صورت می‌پذیرد:

1. **مرحله ۱ (غربالگری متادیتا و وتوی اولیه - زیر ۲ ثانیه):** حذف مخازن آرشیوشده، بدون لایسنس، دارای مجوز ویروسی یا باگ‌های بحرانی بر مبنای متادیتای سبک API.
2. **مرحله ۲ (معاینه فنی عینی و تست‌ها - زیر ۵ ثانیه):** بررسی وجود پوشه‌های تست فعال، سبز بودن بیلد GitHub Actions، کانفیگ‌های لایت و تایپینگ.
3. **مرحله ۳ (کالبدشکافی عمیق معماری - زیر ۱۰ ثانیه):** تحلیل ساختار فایل‌های اصلی، تراکم وابستگی‌ها و شاخص چکش‌خواری بر روی ۳ فینالیست نهایی.

۴. ماتریس امتیازدهی ۵ فاکتوری (Scoring Matrix)

معیار ارزیابی	وزن نهایی	شاخص‌ها و مبانی سنجش فنی
۱. انطباق معماری و چکش‌خواری	**۲۵٪**	قابلیت کپسوله‌سازی ماژولار، سهولت استخراج ورودی/خروجی به صورت DTO، نبود وابستگی به کتابخانه‌های ممنوعه پروژه میزبان و حداقل بودن اصطکاک ریفکتورینگ.
۲. کیفیت و تمیزی کد	**۲۵٪**	سبز بودن بیلد CI/CD، وجود تست‌های واحد معتبر، تایپ‌های صریح و استاندارد، و عدم وجود کدهای درهم‌تنیده.
۳. سلامت و بازخورد جامعه	**۲۵٪**	نرخ بسته‌شدن ایشوها، سرعت پاسخگویی توسعه‌دهندگان به باگ‌ها، تاریخ آخرین کامیت (عدم رهاشدگی پروژه) و پویایی کامیونیتی.
۴. محبوبیت و اعتبار مخزن	**۱۵٪**	تعداد ستاره‌ها (با نرمال‌سازی لگاریتمی جهت جلوگیری از فریب Star Hype)، تعداد فورک‌ها و اعتبار ناشر.
۵. پاکیزگی وابستگی‌ها و لایسنس	**۱۰٪**	سبک بودن درخت پکیج‌های وابسته (عدم تورم node_modules یا venv) و لایسنس‌های تجاری آزاد با محافظت پتنت (/ Apache 2.0 MIT).

قاعده وتوی مطلق (Hard Veto Gate):

پروژه‌هایی که دارای لایسنس‌های ناسازگار با اهداف تجاری (نظیر AGPL یا GPL v3) باشند یا آسیب‌پذیری‌های بحرانی شناخته‌شده (Critical CVE) در آن‌ها گزارش شده باشد، **نمره صفر** دریافت کرده و فوراً از فهرست پیشنهادات حذف می‌شوند.

۵. برتری‌ها و مزایای رقابتی

- **سرعت فوق‌العاده:** دستیابی به ارزیابی کامل مخازن در بازه زمانی ۸ تا ۱۲ ثانیه.
- **پروتکل ضد تملق (Zero-Sycophancy):**** سیستم از تایید کورکورانه اشتباهات اپراتور پرهیز کرده و عواقب فنی را مستند یادآوری می‌کند، ضمن آنکه تصمیم نهایی با اپراتور است.
- **اصطکاک صفر (Zero-Friction UX):** پرهیز از سؤال‌پیچ کردن مکرر اپراتور؛ ارائه پاسخ یکپارچه بصری با نشان‌های Shields.io، جداول منظم و کادرهای بازشونده.
- **کپسوله‌سازی و استقلال:** کدهای برگزیده مستقیماً رها نمی‌شوند، بلکه با تولید آداپتورهای اختصاصی ایزوله شده و تضمین تعویض‌پذیری در آینده را دارند.
- **توسعه مهندسی‌شده با تست ۱۰۰٪:** عبور موفق از ۳۰۶ تست واحد در تمام لایه‌ها.

۶. نقاط کور و محدودیت‌های سیستم (Blind Spots)

۱. محدودیت نرخ درخواست گیت‌هاب (GitHub API Rate Limits):

گیت‌هاب برای درخواست‌های بدون لاگین سقف ۶۰ درخواست در ساعت و با توکن شخصی سقف ۵,۰۰۰ درخواست در ساعت دارد. برای پروژه‌های بزرگ سازمانی ذخیره Personal Access Token در تنظیمات الزامی است.

۲. ریسک حملات زنجیره تأمین نوظهور (Zero-Day Malicious Packages):

فیلترهای امنیتی به پایگاه‌های شناخته‌شده آسیب‌پذیری (مانند CVE و Dependabot) وابسته‌اند. اگر پکیجی ساعاتی پیش هدف تزریق کد مخرب قرار گرفته باشد و هنوز در پایگاه‌ها ثبت نشده باشد، با اسکن متادیتا قابل شناسایی نیست و نیازمند بررسی در یک Sandbox امنیتی است.

۳. چالش مونوریپوها (Monorepos):

در پروژه‌های بزرگی که ده‌ها پکیج مستقل درون پوشه packages/ دارند، کانفیگ‌های تست و متادیتای گیت‌هاب برای کل مخزن ثبت شده است. اسکیل ممکن است در سنجش یک زیرپکیج خاص، دچار تقریب در نمره تمیزی کد شود.

۴. لایسنس‌های تودرتو و وابستگی‌های گذرا (Transitive Dependencies):

گاهی مخزنی لایسنس رسمی MIT دارد، اما درون درخت وابستگی‌های خود از کتابخانه‌ای با لایسنس AGPL استفاده کرده است. اسکیل لایسنس رسمی سطح مخزن را بررسی می‌کند و کشف لایسنس‌های گذرا نیازمند ایجاد فایل جامع SBOM است.
